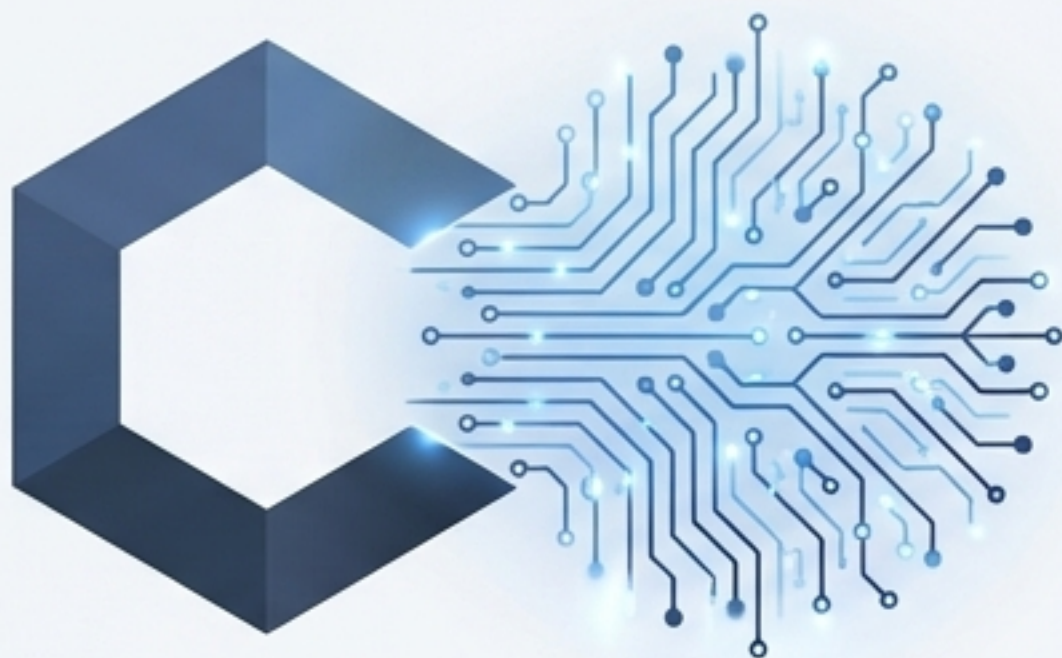




DeepCode: ก้าวกระโดดครั้งใหญ่ จากผู้ช่วยเขียนโค้ดสู่วิศวกรซอฟต์แวร์อัตโนมัติ

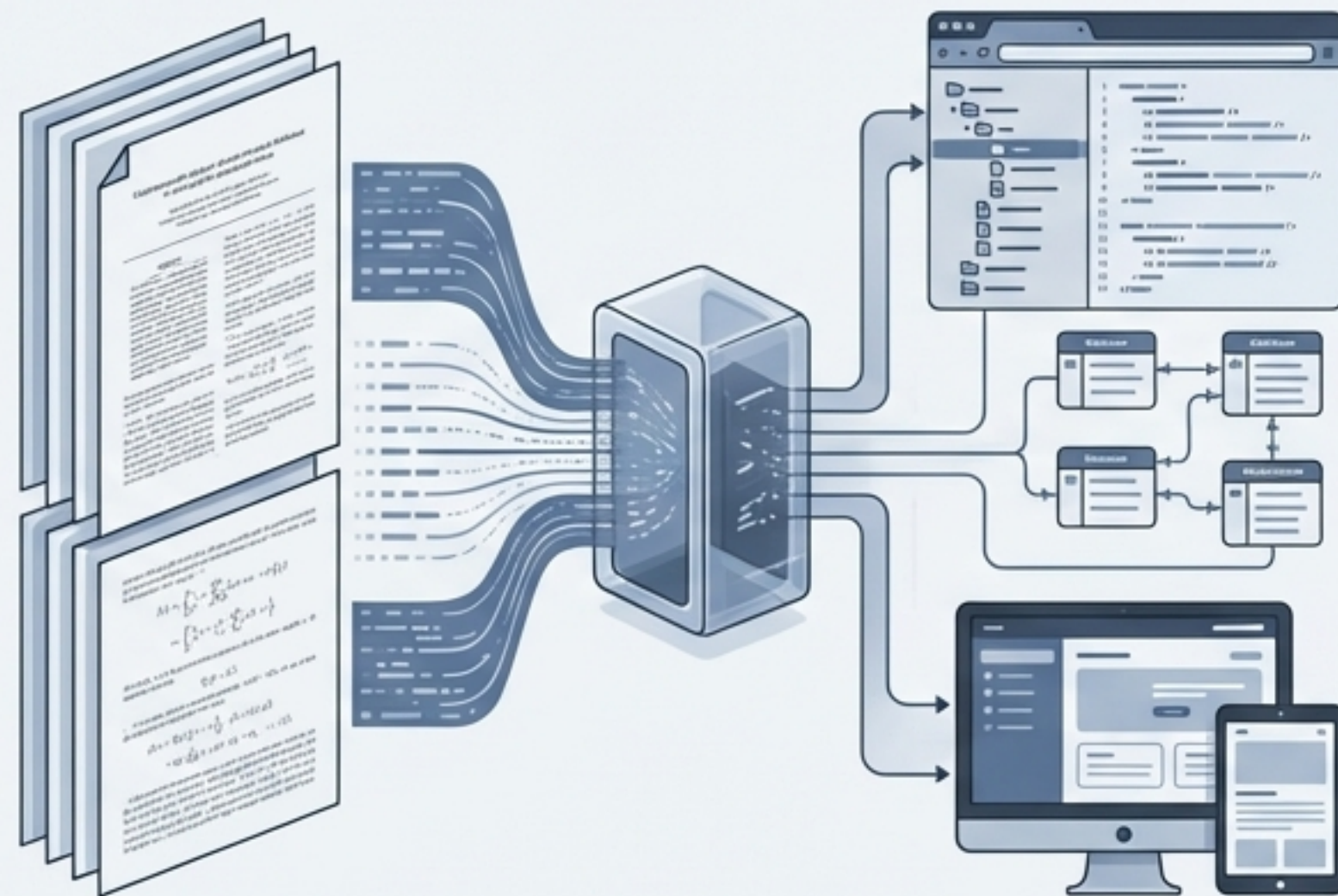
ปฏิบัติการสังเคราะห์โค้ดเบสจากเอกสาร
ด้วยการจัดการกระแสข้อมูลอย่างมีประสิทธิภาพ

วิสัยทัศน์สูงสุด: วิศวกรรมซอฟต์แวร์อัตโนมัติเต็มรูปแบบ

โลกของการพัฒนาซอฟต์แวร์กำลังเปลี่ยนผ่านจากเครื่องมือ AI ที่เป็นเพียง "ผู้ช่วย" (AI-assisted coding) ซึ่งทำหน้าที่แค่เติมโค้ดหรือแนะนำฟังก์ชันไปสู่ยุคใหม่ของ "วิศวกรรมซอฟต์แวร์ที่ขับเคลื่อนโดย Agent" (Agentic Software Engineering)

ในระบอบทัศน์ใหม่นี้ AI Agent จะไม่ได้เป็นแค่ผู้ช่วยแต่จะทำหน้าที่เป็น "วิศวกร" ที่สามารถวางแผนสถาปัตยกรรม และสร้างโปรเจกต์ซอฟต์แวร์ทั้งระบบได้จากข้อกำหนดระดับสูง เช่น เอกสารทางเทคนิค

บททดสอบที่ทำทายที่สุดสำหรับวิสัยทัศน์นี้คือการเปลี่ยนเอกสารงานวิจัยทางวิทยาศาสตร์คือการเปลี่ยนเอกสารงานวิจัยทางวิทยาศาสตร์ที่ซับซ้อนให้กลายเป็นโค้ดเบสที่สมบูรณ์และทำงานได้จริง (Document-to-Repository Synthesis)



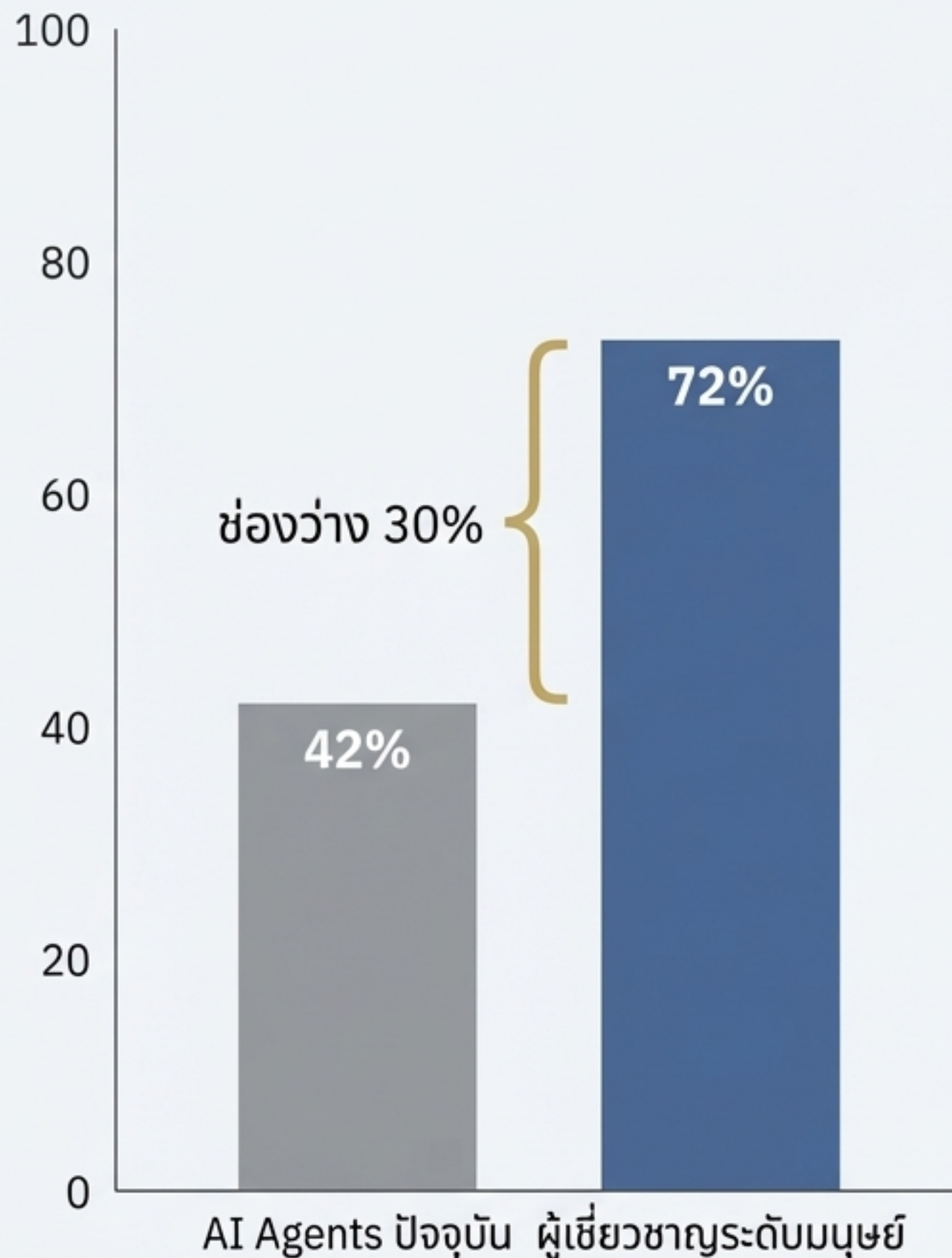
ความเป็นจริง: ช่องว่างด้านประสิทธิภาพที่ยังห่างไกล

แม้จะมีวิสัยทัศน์ที่ยิ่งใหญ่ แต่เทคโนโลยี AI Agent ในปัจจุบันยังคงล้มเหลวในการทำงานที่ซับซ้อนนี้

จากการทดสอบบนชุดข้อมูลมาตรฐาน PaperBench ซึ่งวัดความสามารถในการสร้างโค้ดจากงานวิจัย ML พบว่า:

- **AI Agents ปัจจุบัน** (เช่น GPT-4o, Claude 3.5) ทำคะแนนความสำเร็จในการสร้างโค้ดได้เพียง **~42%**
- ในขณะที่ **ผู้เชี่ยวชาญระดับมนุษย์** (นักศึกษาระดับปริญญาเอกสาขา ML จากสถาบันชั้นนำ) ทำได้ถึง **~72%**

ช่องว่าง 30% นี้ชี้ให้เห็นถึงข้อจำกัดพื้นฐานของแนวทางปัจจุบัน และเป็นอุปสรรคสำคัญที่ขัดขวางการไปถึงวิศวกรรมซอฟต์แวร์อัตโนมัติอย่างแท้จริง

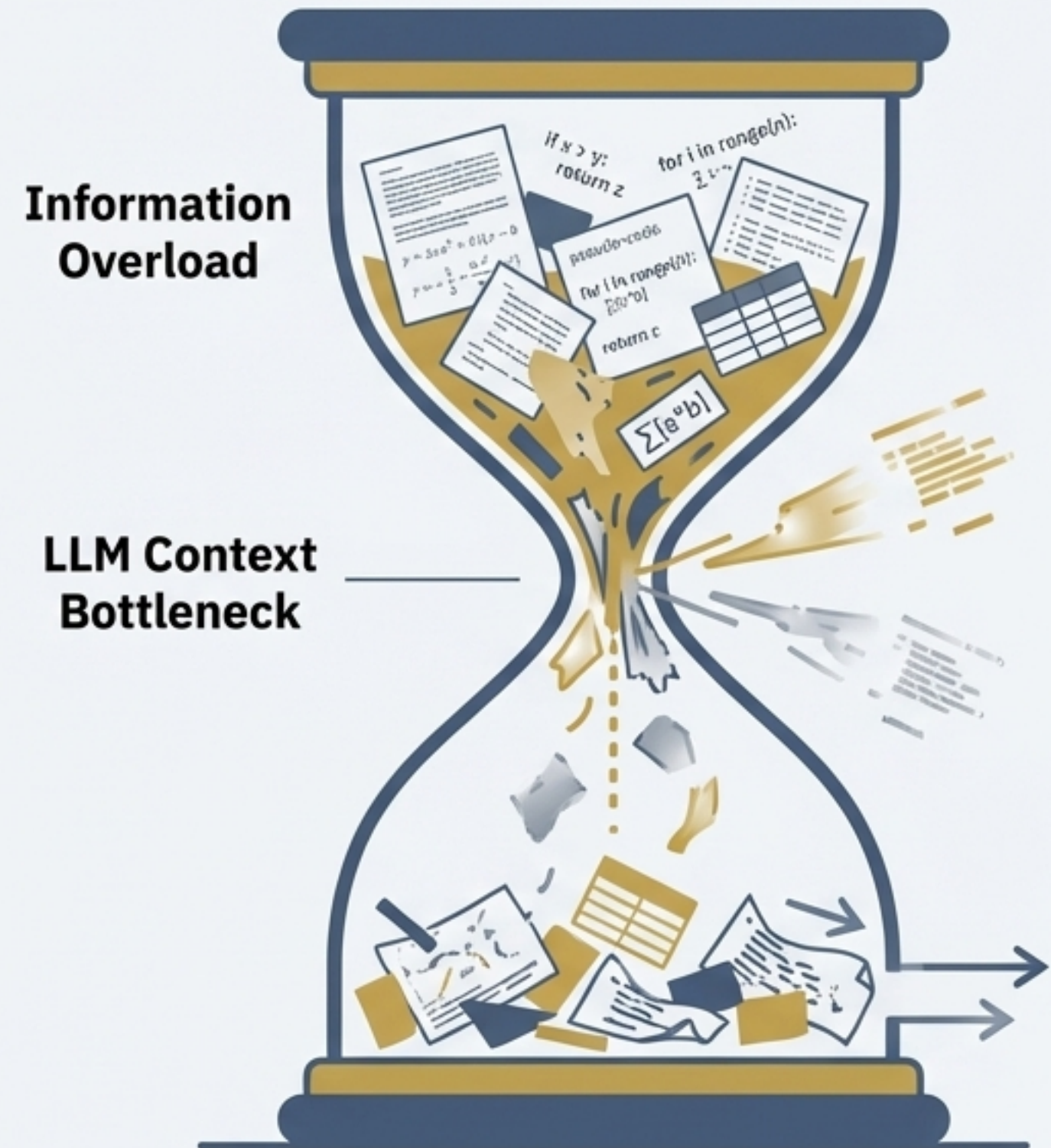


ต้นตอของความล้มเหลว: ข้อมูลมหาศาลปะทะคอขวดของ Context

ความล้มเหลวของ AI Agent ไม่ได้เกิดจากความสามารถในการเขียนโค้ด แต่เกิดจากความขัดแย้งพื้นฐานระหว่าง “ข้อมูลที่ล้นเกิน” (Information Overload) กับ “ข้อจำกัดด้าน Context ของ LLM” (LLM Context Bottleneck)

การพยายามป้อนข้อมูลดิบทั้งหมดเข้าไปในโมเดลเปรียบเสมือนการเทข้อมูลจำนวนมากลงในช่องทางที่คับแคบ ทำให้ “สัญญาณ” (ข้อมูลสำคัญทางอัลกอริทึม) ถูกบดบังด้วย “สัญญาณรบกวน” (ข้อมูลที่ไม่จำเป็น)

สิ่งนี้ทำให้ Signal-to-Noise Ratio (อัตราส่วนสัญญาณต่อสัญญาณรบกวน) พังทลายลง ส่งผลให้โมเดลไม่สามารถรักษาความถูกต้องและความสอดคล้องของโปรเจกต์ได้

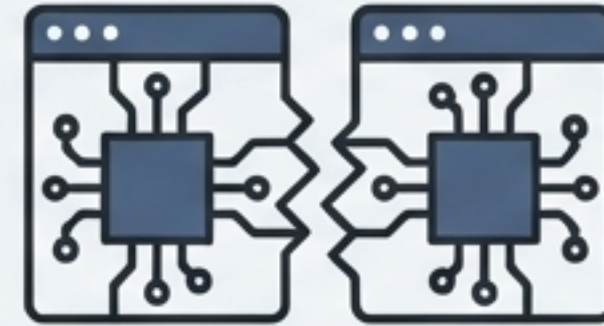


คอบวณนี้ก่อให้เกิดความล้มเหลวหลัก 4 ประการ



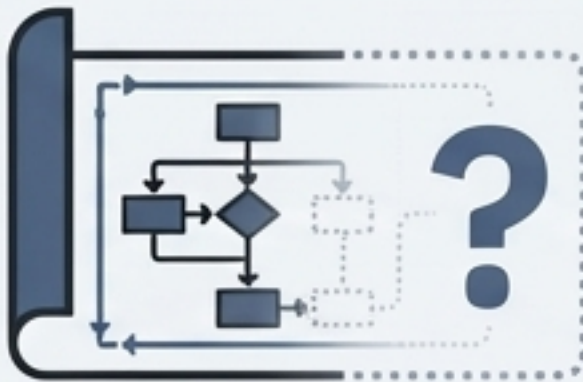
1. Specification Preservation (การรักษาข้อกำหนด)

ไม่สามารถรักษาความเชื่อมโยงที่ถูกต้องระหว่างข้อกำหนดที่กระจัดกระจายในเอกสารกับโค้ดที่สร้างขึ้นได้



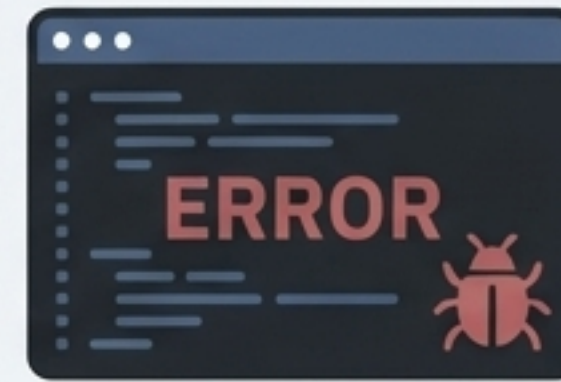
2. Global Consistency (ความสอดคล้องของโค้ดทั้งระบบ)

การสร้างโค้ดที่ละไฟล์ภายใต้ Context ที่จำกัด ทำให้เกิดความไม่สอดคล้องกันระหว่างโมดูลต่างๆ เช่น interface หรือ type ที่ไม่ตรงกัน



3. Underspecified Design (การเติมเต็มดีไซน์ที่ไม่สมบูรณ์)

เอกสารมักจะระบุแค่อัลกอริทึมหลัก แต่ละเว้นรายละเอียดการ บล็อกิเทศ (implementation) ที่สำคัญ ซึ่ง AI ไม่สามารถอนุมานได้อย่างถูกต้อง



4. Executable Faithfulness (ความถูกต้องในการทำงานจริง)

โค้ดที่สร้างขึ้นอาจดูเหมือนถูกต้อง แต่เต็มไปด้วยบั๊กเชิงตรรกะ, dependency ที่ขัดแย้งกัน, หรือ pipeline ที่เปราะบาง ทำให้ไม่สามารถรันได้จริง

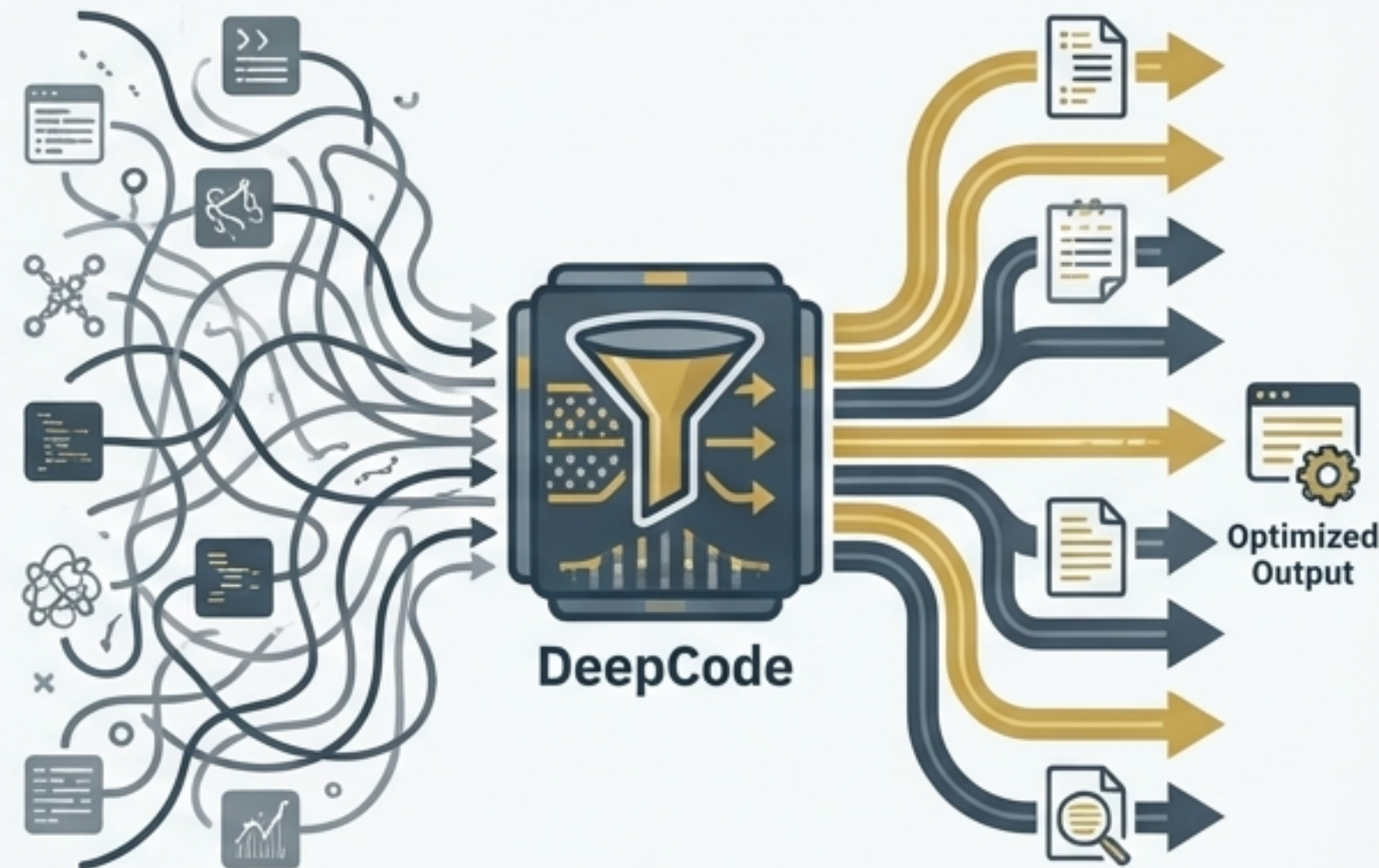
พลิกกระบวนทัศน์ใหม่: จัดการการสังเคราะห์โค้ดเสมือน การปรับช่องสัญญาณให้ดีที่สุด

DeepCode เสนอแนวทางแก้ปัญหาที่ไม่ใช่การเพิ่มขนาด Context Window แต่เป็นการเปลี่ยนกระบวนทัศน์ (Paradigm Shift) โดยสิ้นเชิง

เรามองว่าการสังเคราะห์โค้ดเบสเป็น "**ปัญหาการปรับช่องสัญญาณให้เหมาะสมที่สุด**" (Channel Optimization Problem)

แทนที่จะป้อนข้อมูลดิบเข้าไปได้การควบคุม ระบบจะต้อง "**จัดการกระแสข้อมูลอย่างมีหลักการ**" (Principled Information-Flow Management)

เป้าหมายคือการ **เพิ่มความหนาแน่นของสัญญาณที่เกี่ยวข้องกับงานให้สูงสุด (Maximize Task-Relevant Signals)** ในขณะที่ **ลดสัญญาณรบกวนที่ไม่จำเป็น (Suppress Irrelevant Noise)** ภายใต้ข้อจำกัดของ Context ที่มีอยู่



หัวใจของ DeepCode: 4 ยุทธศาสตร์การจัดการข้อมูล



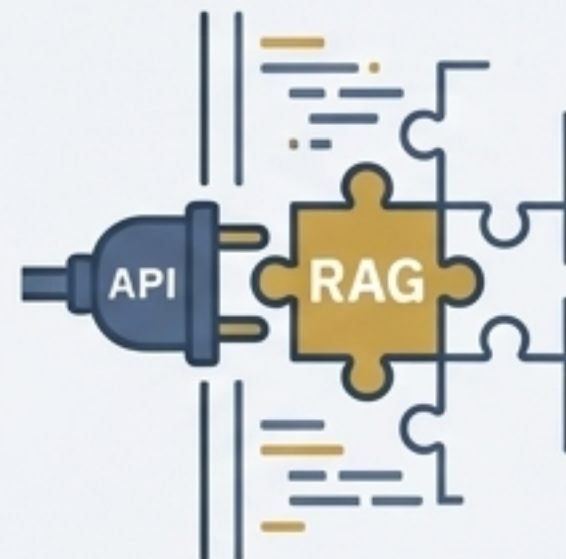
1. Source Compression (การบีบอัดข้อมูลต้นทาง)

กลั่นกรองเอกสารต้นฉบับที่ไม่มีโครงสร้างให้กลายเป็น "พิมพ์เขียว" (Blueprint) ที่มีความหนาแน่นของข้อมูลสำคัญสูง



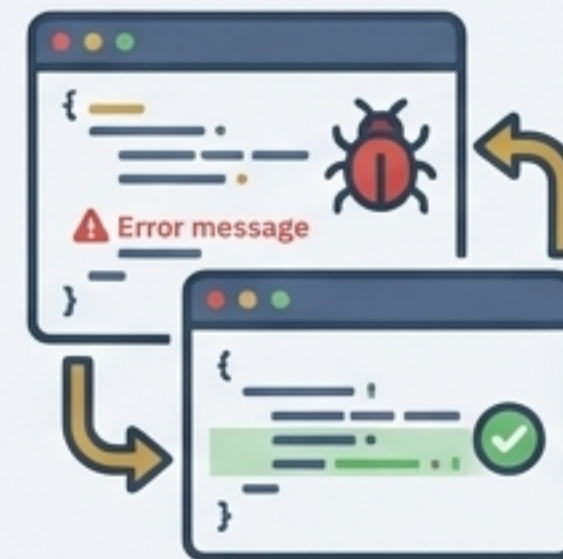
2. Structured Indexing (การจัดทำดัชนีเชิงโครงสร้าง)

สรุปสถานะของโค้ดเบสที่กำลังพัฒนาให้เป็น "หน่วยความจำ" (Memory) ที่มีโครงสร้าง เพื่อรักษาความสอดคล้องของทั้งระบบโดยไม่ทำให้ Context ล้น



3. Knowledge Injection (การเติมความรู้ตามเงื่อนไข)

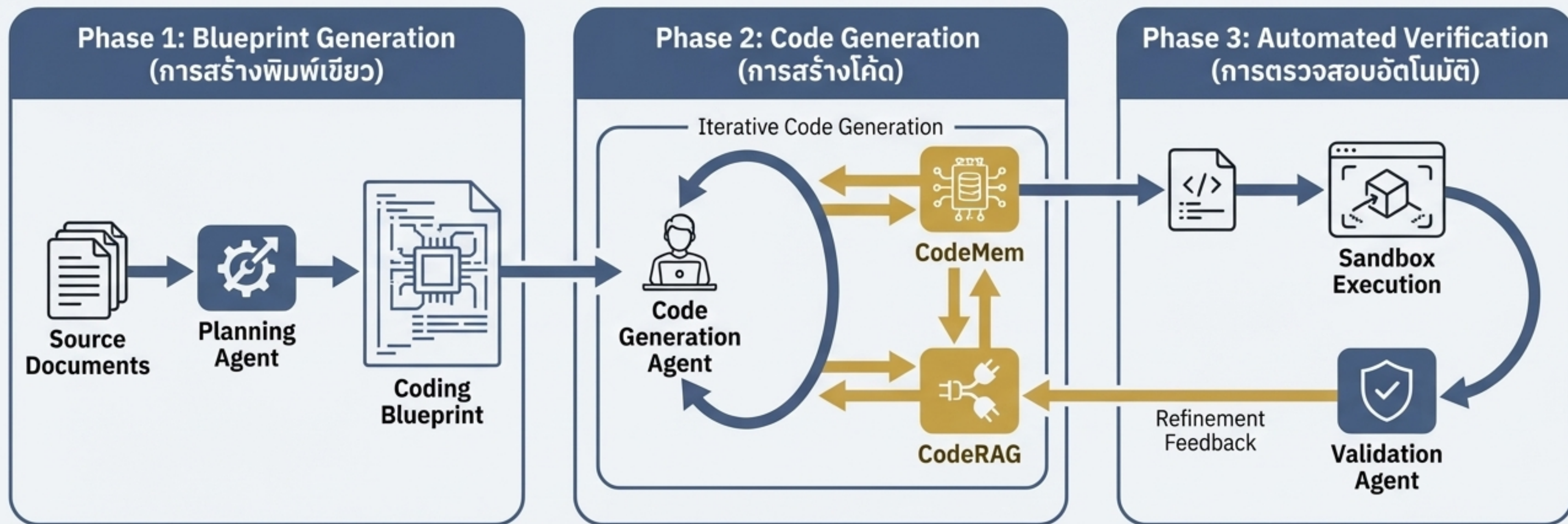
ใช้ Retrieval-Augmented Generation (RAG) เพื่อดึงรูปแบบการ บล็อกโค้ด ที่เป็นมาตรฐานจากภายนอกมาเติมเต็มช่องว่างของความรู้ที่ไม่ได้ระบุไว้ในเอกสาร



4. Error Correction (การแก้ไขข้อผิดพลาดแบบวงปิด)

ใช้ผลลัพธ์จากการรันโค้ดจริง (Execution Feedback) เป็นสัญญาณในการแก้ไขข้อบกพร่องเพื่อให้แน่ใจว่าโค้ดที่ได้สามารถทำงานได้ถูกต้อง

สถาปัตยกรรม DeepCode: กรอบการทำงาน 3 เฟสเพื่อ การสร้างเคราะห์โค้ดที่สมบูรณ์แบบ



Planning Agent จะทำหน้าที่ “บีบอัดข้อมูลต้นทาง” โดยเปลี่ยนเอกสารวิจัยให้เป็นพิมพ์เขียวการ บล็อกอิเทสที่มีโครงสร้างชัดเจนและความหนาแน่นของสัญญาณสูง

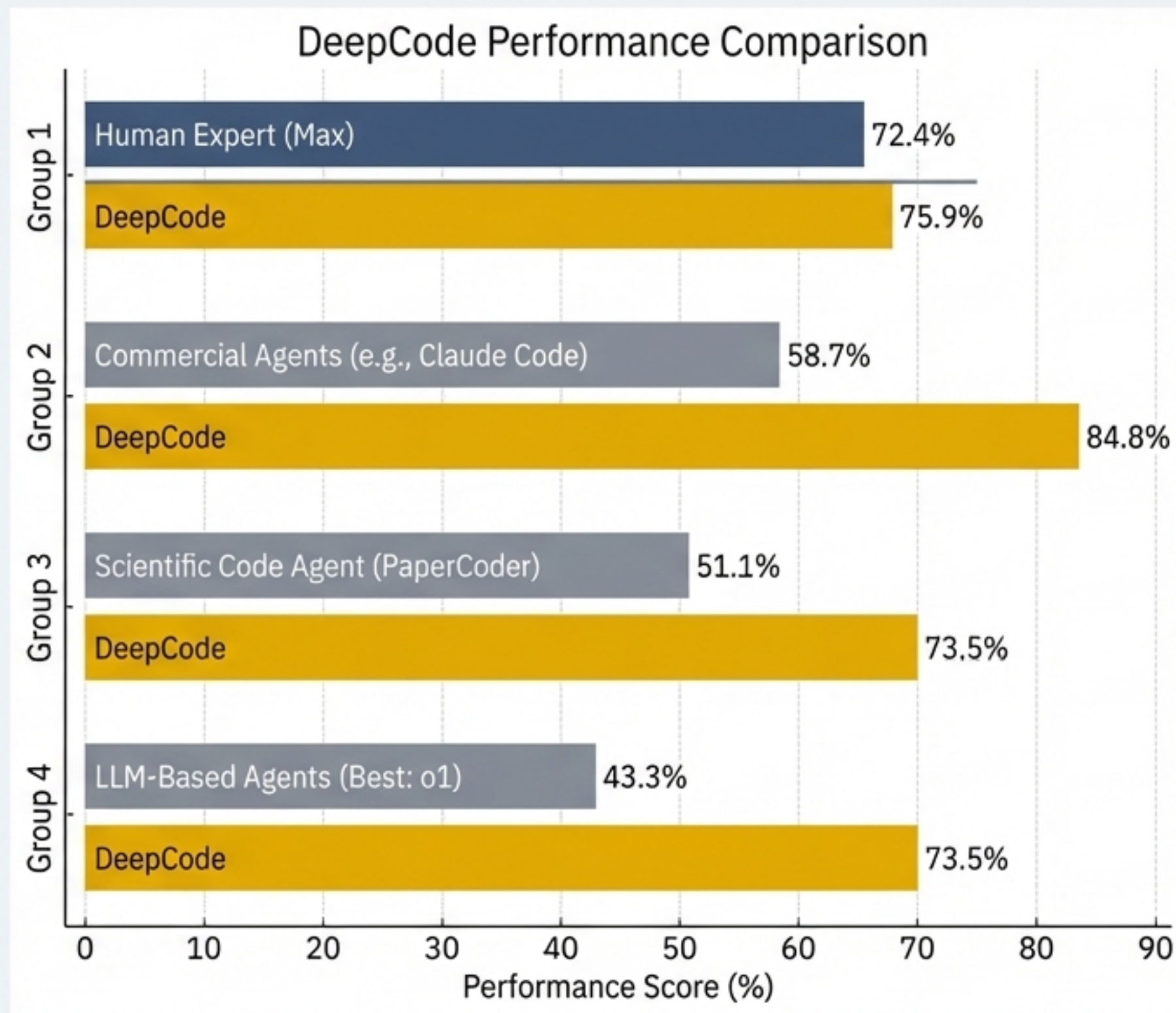
Code Generation Agent จะสร้างไฟล์โค้ดตามพิมพ์เขียวโดยใช้ **CodeMem** เพื่อรักษาความสอดคล้องของโปรเจกต์ และ **CodeRAG** เพื่อดึงความรู้จากภายนอกมาช่วย

Validation Agent จะทำหน้าที่ “แก้ไขข้อผิดพลาดแบบวงปิด” โดยทำการวิเคราะห์และรันโค้ดใน Sandbox เพื่อหาและแก้ไขข้อผิดพลาดอัตโนมัติ

ผลลัพธ์: DeepCode มีประสิทธิภาพเหนือกว่าผู้เชี่ยวชาญระดับมนุษย

DeepCode สร้างมาตรฐานใหม่ (State-of-the-Art) ในการสังเคราะห์โค้ดจากเอกสาร โดยมีประสิทธิภาพเหนือกว่าทุกเกณฑ์มาตรฐานอย่างมีนัยสำคัญ

- **เหนือกว่าผู้เชี่ยวชาญมนุษย์:** ทำคะแนนเฉลี่ยได้ **75.9%** เทียบกับคะแนนสูงสุดของมนุษย์ที่ **72.4%**
- **เหนือกว่า Commercial Agents:** ทำคะแนนเฉลี่ย **84.8%** เาชนะ Claude Code (58.7%)
- **เหนือกว่า Scientific Code Agents:** เาชนะ PaperCoder ซึ่งทำคะแนนได้ **51.1%**
- **เหนือกว่า LLM-Based Agents:** มีประสิทธิภาพสูงกว่า Agent ที่ดีที่สุด (o1) ซึ่งทำคะแนนได้ **43.3%**



เจาะลึกประสิทธิภาพ: สถาปัตยกรรมคือปัจจัยแห่งชัยชนะ

****ข้อสังเกตที่สำคัญที่สุด**:** DeepCode, Cursor, และ Claude Code ใช้โมเดลภาษาพื้นฐานเดียวกัน (Claude Sonnet 4.5-thinking) แต่ประสิทธิภาพของ DeepCode กลับสูงกว่าอย่างมหาศาล. สิ่งนี้เป็นข้อพิสูจน์ที่ชัดเจนว่า **ความได้เปรียบของ DeepCode ไม่ได้มาจากพลังของ LLM เพียงอย่างเดียว แต่มาจากสถาปัตยกรรม Agent ที่เหนือกว่า**

Model	fre	rice	bam	pinn	mech-u	Avg.
Codex (GPT 5 Codex-high)	0.4095	0.3645	0.1937	0.5382	0.4926	0.3997
Claude Code (Claude Sonnet 4.5-think)	0.6286	0.3787	0.3829	0.7233	0.8222	0.5871
Cursor (Claude Sonnet 4.5-think)	0.6344	0.4186	0.3779	0.7748	0.7148	0.5841
DeepCode (Claude Sonnet 4.5-think)	0.8435	0.7380	0.8530	0.9474	0.8888	0.8541

พิสูจน์คุณค่าของทุกองค์ประกอบ: การทดลองแยกส่วน

1. CodeMem (Structured Indexing)

การใช้หน่วยความจำที่มีโครงสร้างช่วยเพิ่มคะแนนจาก 0.33-0.43 เป็น **0.70-0.92** ในงานที่ซับซ้อน ป้องกันการสูญเสียข้อมูลสำคัญเมื่อ Context เต็ม



2. CodeRAG (Knowledge Injection)

สำหรับโมเดลขนาดเล็ก (Gemini-2.5-Flash) การใช้ CodeRAG ช่วยเพิ่มประสิทธิภาพได้ถึง **+70%** โดยการเติมเต็มช่องว่างความรู้ที่โมเดลไม่มี



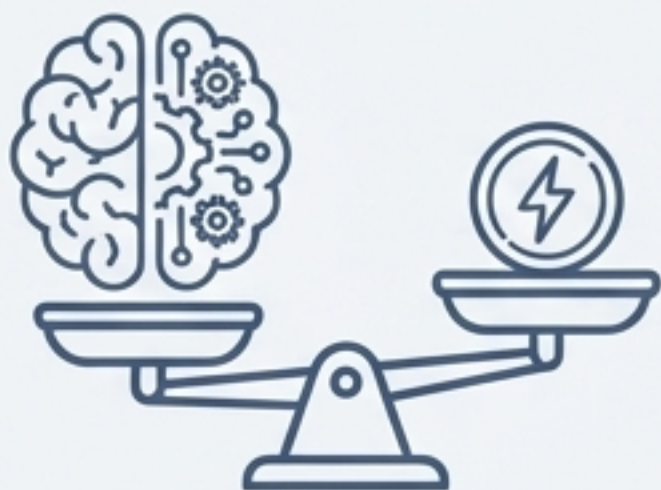
3. Automated Verification (Error Correction)

แม้จะเป็นขั้นตอนสุดท้าย แต่ช่วยเพิ่มคะแนนได้ **3.7%-6.5%** โดยการแก้ไขบักเล็กๆ น้อยๆ ที่ส่งผลกระทบต่อารันโค้ด



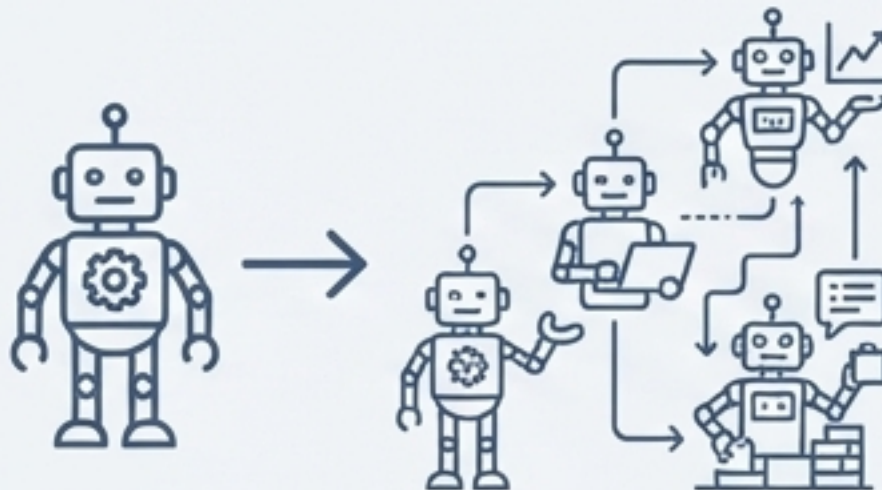
ผลการทดลองยืนยันสมมติฐานหลักของเรา: **การจัดการกระแสข้อมูลอย่างมีประสิทธิภาพ คือกุญแจสำคัญสู่ความสำเร็จ**

อนาคตของวิศวกรรมซอฟต์แวร์: 3 ความท้าทายสู่การเป็นวิศวกร AI ที่สมบูรณ์



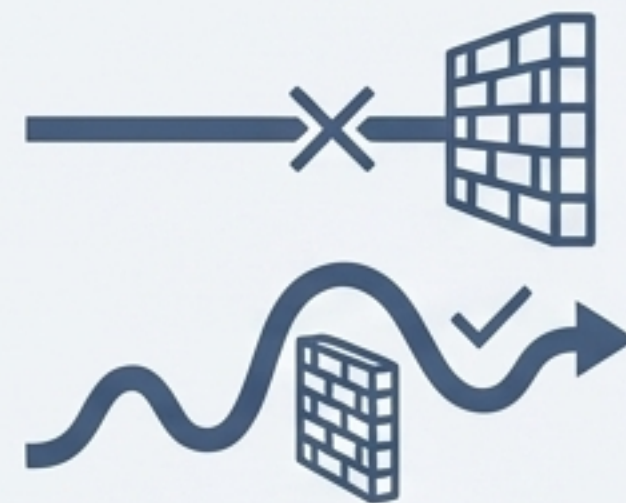
1. Agentic Capability vs. Computational Efficiency

ทำอย่างไรให้ Agent ที่มีความสามารถสูงทำงานบนโมเดลขนาดเล็กและประหยัดต้นทุนได้ โดยไม่ต้องพึ่งพาโมเดลขนาดใหญ่ที่มีราคาแพง



2. From Episodic to Evolving Agents

ทำอย่างไรให้ Agent สามารถสะสมประสบการณ์และความรู้จากโปรเจกต์หนึ่งไปใช้อีกโปรเจกต์หนึ่งได้ เหมือนกับการเติบโตของมนุษย์



3. Dynamic Planning and Adaptability

ทำอย่างไรให้ Agent สามารถปรับเปลี่ยนแผนการทำงานกลางคันได้ เมื่อเจอปัญหาที่ไม่คาดคิดระหว่างการเขียนโค้ด

DeepCode: วางรากฐานใหม่ สำหรับระบบอัตโนมัติที่แข็งแกร่ง

DeepCode พิสูจน์ให้เห็นว่าคุณจะสำคัญสู่วิศวกรซอฟต์แวร์
อัตโนมัติที่แข็งแกร่ง ไม่ใช่แค่การขยาย Context Window
ให้ใหญ่ขึ้นอย่างไร้ขีดจำกัด แต่คือ **การจัดการกระแส
กระแสนข้อมูลอย่างมีหลักการ** (Principled
Information-Flow Management)

ด้วยการปรับช่องสัญญาณข้อมูลให้เหมาะสมที่สุดผ่านการ
กลั่นกรองพิมพ์เขียว, การใช้หน่วยความจำเชิงโครงสร้าง,
การเติมความรู้จากภายนอก, และการแก้ไขข้อผิดพลาด
แบบวงปิด ทำให้ DeepCode สามารถสร้างมาตรฐานใหม่
(SOTA) และมีประสิทธิภาพเหนือกว่าผู้เชี่ยวชาญ
ระดับมนุษย์ได้

แนวทางนี้ไม่เพียงแต่แก้ปัญหาการสร้างโค้ดจากเอกสาร แต่
ยังวางรากฐานที่สำคัญสำหรับอนาคตของการค้นพบทาง
วิทยาศาสตร์อัตโนมัติและการพัฒนาซอฟต์แวร์ที่เชื่อถือได้

